

BangSquad

프로젝트 기술 문서

정예원



게임명	BangSquad
프로젝트 유형	기업협약 팀프로젝트
장르	3D 협동 액션 RPG
인원	개발 4인 / 기획 1인 (총 5인)
기간	2025.12.22 ~ 2026.03.06 (75일)
개발 환경	Unreal Engine 5, Rider, GitHub
기술 스택	UE5, C++, OnlineSubsystem, Replication, RPC, DataAsset, Gameplay Framework
담당 기능	멀티플레이 세션, 로비, Stage, 리스폰, 체크포인트, 관전

게임 소개



역할을 나누어 스테이지와 기믹을 돌파하는 Listen Server 기반 4인 협동 액션 RPG

- Listen Server 기반 4인 멀티플레이
- 로비 → Stage → MiniGame → Boss 구조
- 직업/상태를 동기화하며 협동 플레이

담당 기능

멀티플레이

- Listen Server 기반 세션 생성 및 참가 구현
- LAN / Steam OnlineSubsystem 분기 설계
- Ready 상태 및 플레이어 목록 동기화
- 서버 권한 기반 직업 선택

로비

- RepNotify 기반 Phase 동기화
- 서버 권한 직업 선택 및 중복 방지
- Ready 상태 및 플레이어 목록 동기화

스테이지

- Stage 전환 및 Portal 시스템 구현
- SaveInterface 기반 객체 저장 및 복원
- Respawn · Observer 시스템 구현

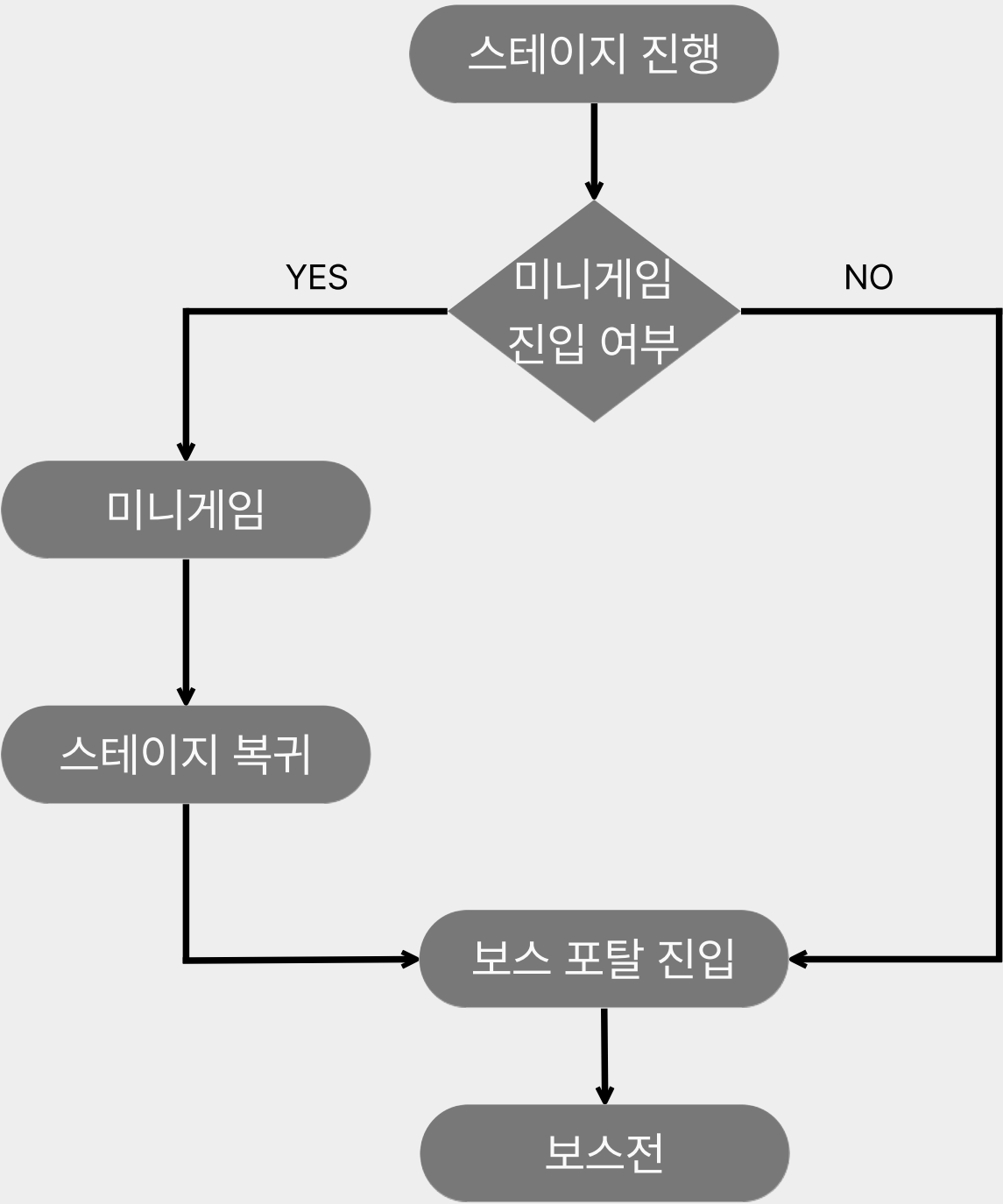
미니게임

- Arena Phase 기반 진행 흐름 구현
 - 실시간 순위 시스템 구현
 - 결과 집계 및 UI 반영
-

메인 흐름



스테이지 진행



게임 규칙 및 상태 관리 구조

플레이어별 상태 저장

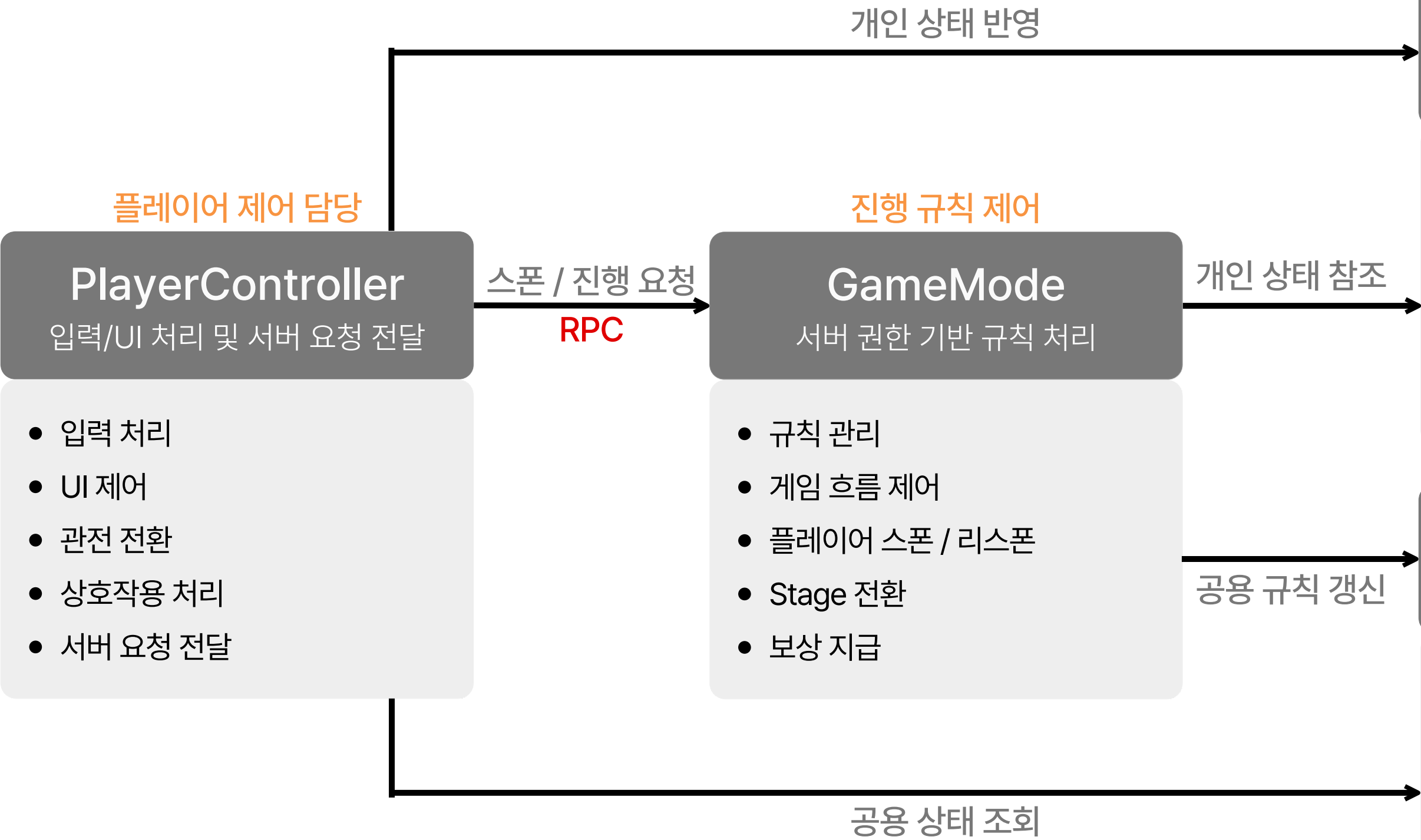
PlayerState
개인 상태 복제 및 Late Join 대응

- 개인 상태 관리
- 직업 정보
- Stage / MiniGame 리스폰 정보
- MiniGame 진행도 / 순위
- QTE 입력 상태

공유 상태 저장

GameState
공용 상태 복제 및 전체 동기화

- 공용 상태 관리
- Stage 공용 체크포인트
- Boss Stage 공용 부활카운트
- MiniGame 진행 상태
- 전체 플레이어 공용 정보



플레이어 제어 담당

PlayerController
입력/UI 처리 및 서버 요청 전달

- 입력 처리
- UI 제어
- 관전 전환
- 상호작용 처리
- 서버 요청 전달

진행 규칙 제어

GameMode
서버 권한 기반 규칙 처리

- 규칙 관리
- 게임 흐름 제어
- 플레이어 스폰 / 리스폰
- Stage 전환
- 보상 지급



BSGameInstance		
- 세션 생성 - CreateSession - OnCreateSessionComplete - ServerTravel	- 세션 검색 - FindSessions - OnFindSessionComplete - SetServerList	- 세션 참가 - JoinSession - OnJoinSessionComplete - ClientTravel

```
SessionInterface->OnCreateSessionCompleteDelegates.AddUObject(
    this, &UBSGameInstance::OnCreateSessionComplete);
```

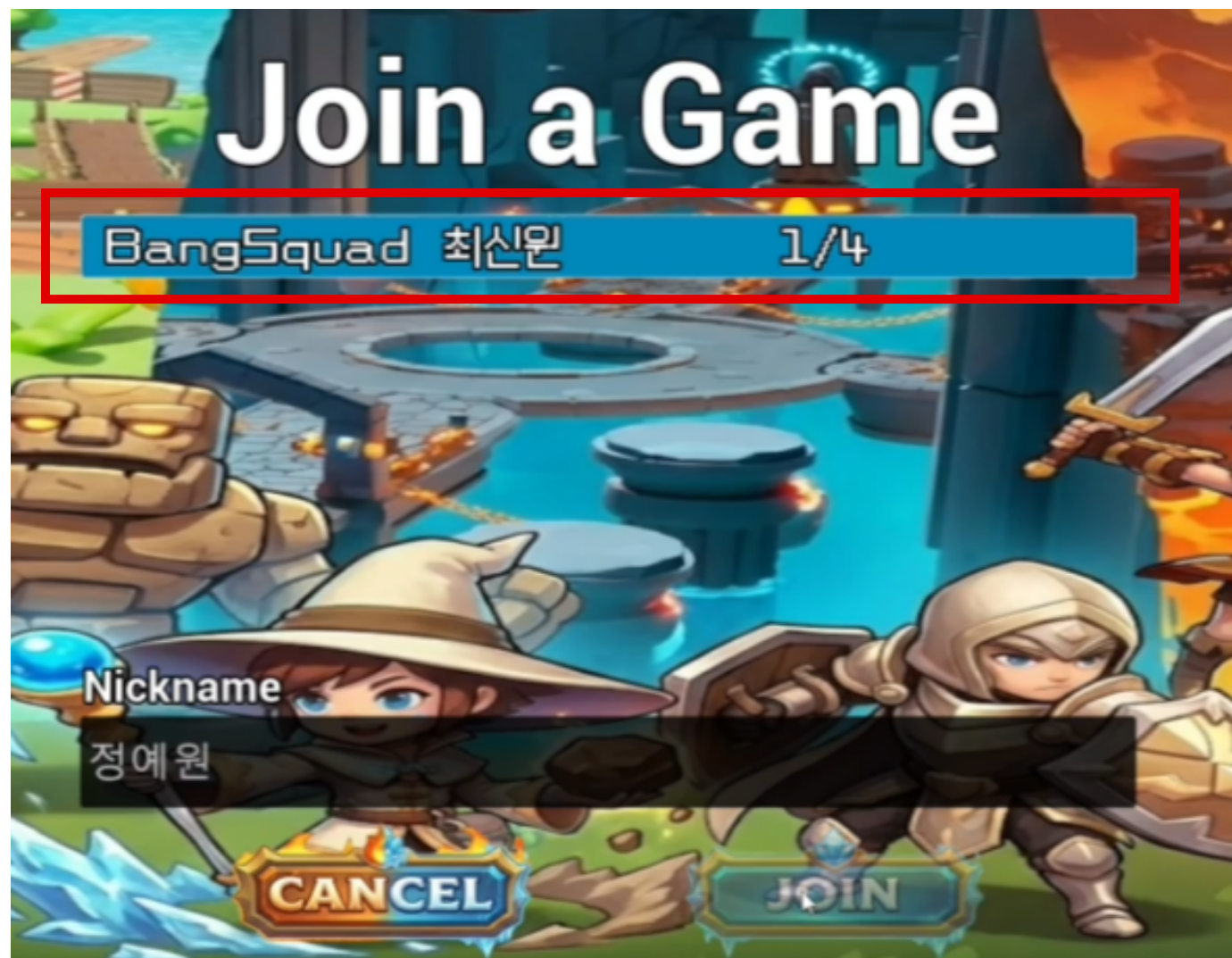
- OnlineSubsystem은 비동기 API이므로, 세션 완료 전에는 Travel 수행 불가능
따라서 완료 Delegate에서만 Travel 및 UI 갱신 수행
- Create / Find / Join Session 완료 후 각각 ServerTravel, 서버 목록 갱신, ClientTravel을 수행하도록 흐름을 구성

	LAN	Steam
blsLANMatch	true	false
bUsesPresence	false	true
검색 방식	Broadcast	Presence
연결 범위	같은 네트워크	Steam 친구



- LAN은 Steam 없이 로컬 멀티플레이 테스트를 지원
- Steam은 Presence 기반 세션 검색과 친구 초대 기능을 지원
- 세션 생성/검색 흐름은 공통으로 유지 OnlineSubsystem 종류에 따라 SessionSettings만 분기하여 테스트 환경 (LAN)과 배포 환경(Steam)을 동일한 코드 구조로 처리

서버 리스트



FOnlineSessionSearchResult
(OnlineSubsystem 내부 타입)

FServerData
(UI 전달용 DTO)

Widget

```
struct FServerData
{
    FString Name;
    uint16 CurrentPlayers;
    uint16 MaxPlayers;
    FString HostUserName;
};
```

- FOnlineSessionSearchResult를 DTO(FServerData)로 변환하여 UI에 직접 노출되지 않도록 구성
- 이를 통해 세션 검색 로직과 서버 목록 UI의 의존성을 분리하고, 표시용 데이터만 전달하도록 설계

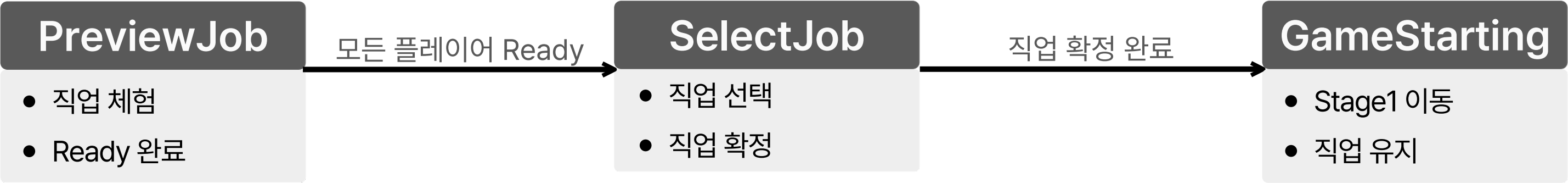
직업 체험 / Ready



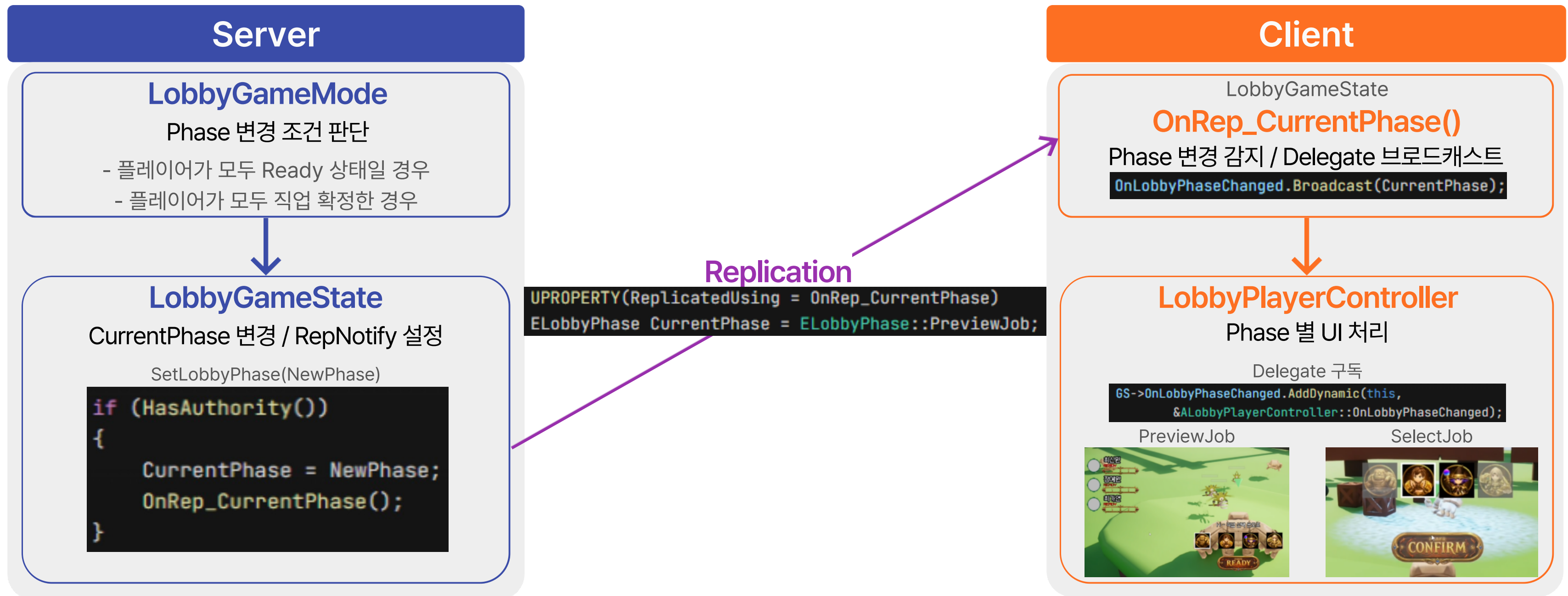
직업 확정



Stage1 진입



- Ready 완료 여부는 서버에서 판단하여 Phase를 전환
- 이벤트 전달이 아닌 상태 복원이 필요했기 때문에, Late Join 지원이 가능한 RepNotify로 동기화
- 직업 정보를 유지한 채 Stage1으로 이동



- 서버의 GameMode가 Phase 변경 조건을 판단하고 GameState의 **CurrentPhase**를 변경
- **CurrentPhase**는 **RepNotify**로 복제되며, **OnRep**에서 Delegate를 Broadcast
- RPC는 이벤트만 전달하므로 Late Join 플레이어는 현재 상태 파악할 수 없음
- UI를 직접 호출하지 않고 Phase 변경 이벤트만 전달해 GameState와 UI의 결합도를 낮춤

직업 선택 UI



서버 검증 후 확정 직업 저장 및 클라이언트 알림

```
GI->SetPlayerJob(FinalJob);  
ServerConfirmedJob(FinalJob);
```



- GameState는 로비 전체의 직업 점유 상태를 관리
- PlayerState는 플레이어별 확정 직업의 기준 데이터를 관리
- GameInstance는 맵 전환 시 로컬 UI 복원용 캐시로 사용
- 공용 상태, 최종 직업, 로컬 복원 캐시를 분리해 직업 선택과 맵 전환을 안정적으로 처리

문제 상황

- 클라이언트 UI 비활성화만으로는 패킷이 도착하기 전 두 플레이어가 **동시에 같은 직업**을 선택할 수 있었다.

기존 방식 한계

- UI 버튼 비활성화
 - 클라이언트 UI만으로는 **서버 최종 상태 보장 불가**
- 서버 기반 확정 직업 목록 관리
 - 직업 변경 시 기존/신규 상태 관리 한계

해결

- 전체 직업 점유 상태는 LobbyGameState에서 관리
- 플레이어별 확정 직업은 LobbyPlayerState에서 분리 관리
- 직업 선택 요청은 LobbyGameMode의 TryConfirmJob()에서 **서버 권한으로 최종 검증**

결과

- 동시 요청에서도 서버 권한으로 **단일 직업만 확정**

```
bool ALobbyGameMode::TryConfirmJob(...)
{
    if (!GS->IsJobAvailable(Job))
        return false;

    if (RequestingPS->GetIsConfirmedJob())
    {
        EJobType OldJob = RequestingPS->GetSavedJobType();
        GS->RemoveTakenJob(OldJob);
    }

    GS->AddTakenJob(Job);
    RequestingPS->SetSavedJobType(Job);
    RequestingPS->SetIsConfirmedJob(true);

    return true;
}
```

트러블슈팅 - SeamlessTravel 생명주기 충돌 해결

문제 상황

- Lobby → Stage SeamlessTravel 중 Access Violation이 간헐적으로 발생
- Stage ↔ MiniGame 이동에서는 정상 동작
- Lobby UI 초기화 과정에서만 불규칙적으로 재현

원인

- GameState가 준비될 때까지 0.2초 반복 Timer로 UI 초기화를 재시도
- SeamlessTravel 중 이전 World 기준 예약 콜백이 뒤늦게 실행되며, 유효하지 않은 World/Controller/UI를 참조

해결

- 반복 Timer 제거
- BeginPlay에서 직접 초기화
- IsValid 검사
- 위젯 수명 관리 보강(RegisterManagedWidget)
- GameStarting 이후 UI 갱신 차단

결과

- Lobby → Stage 전환 시 AccessViolation 제거
- 월드 생명주기에 맞게 UI 초기화 구조 개선
- 반복 Timer 제거로 불필요한 재시도 로직 제거

문제 코드

```
// GameState 복제 완료를 기다리기 위함
GetWorld()->GetTimerManager().SetTimer(
    InitTimerHandle,
    this,
    &ALobbyPlayerController::InitLobbyUI,
    0.2f,
    true
);
```

수정 후 코드

```
// 지연 타이머 제거
InitLobbyUI();
```

안정성 개선 코드

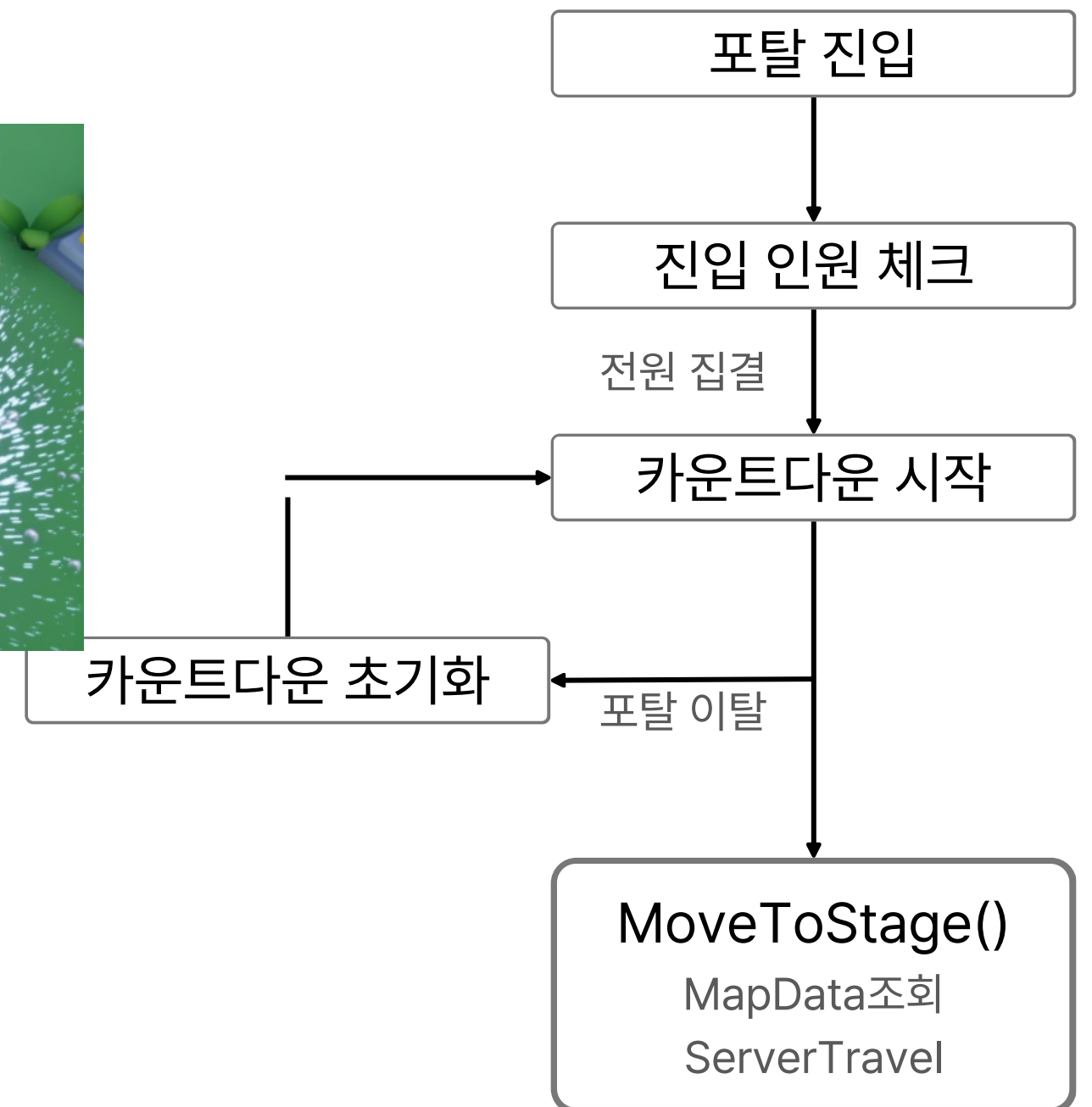
```
if (GS && GS->CurrentPhase == ELobbyPhase::GameStarting)
    return;

if (IsValid(LobbyMainWidget) &&
    LobbyMainWidget->IsInViewport())
{
    LobbyMainWidget->UpdatePlayerList();
}
```

MapDataAsset

Index [3]	6 members
Stage Index	Stage 1
Section	Mini Game
Level	Stage1_MiniGame
Display Name	스테이지1 - 미니게임
Loading Image	Stage1_Loading
Mini Game Result Image	Stage1_MiniGameResultBackground

전원 집결 & 카운트다운



- 플레이어가 포탈에 진입하면 현재 진입 인원을 확인
- 전원 집결 시 카운트다운을 시작하고, 이탈 시 카운트다운을 초기화
- Stage Index / Section으로 이동 대상을 결정하고, DataAsset에서 Level을 조회하여 ServerTravel 수행
- Level/Texture 등 Asset 참조가 필요하여 DataTable 대신 DataAsset으로 구성

FActorSaveData

- ActorTransform
- FloatData
- BoolData

ISaveInterface

- SaveID
- SaveActorData
- LoadActorData

```
UGameplayStatics::GetAllActorsWithInterface(GetWorld(), USaveInterface::StaticClass(), PuzzleActors);  
for (AActor* Actor : PuzzleActors)  
{  
    SaveInterface->SaveActorData(TempData);  
    GI->SaveDataToInstance(SaveInterface->GetSaveID(), TempData);  
}
```

- 미니게임 이후 스테이지로 복귀할 때 퍼즐, 오브젝트 상태가 초기화되는 문제 해결
- 맵 이동 전 ISaveInterface를 구현한 객체를 수집해 상태 저장
- **ISaveInterface를 통해** 새로운 저장 대상 추가 시 기존 저장 코드 수정 없이 확장 가능하도록 설계
- 저장 데이터는 SaveID 기준으로 GameInstance에 보관하고 재진입 시 복원
- 저장 시스템은 인터페이스 함수만 호출하고, 실제 저장 데이터 구성은 각 객체가 담당

ISaveInterface 대상 수집

오브젝트 상태 저장

(SaveActorData() 호출)

SaveID 기준 상태 저장

(GameInstance)

스테이지 재진입

저장 데이터 조회

오브젝트 상태 복원

(LoadActorData() 호출)

문제 상황

- MiniGame 복귀 후 일부 클라이언트에서 RisingPlatform 복원 실패
- 다른 저장 액터는 정상 복원
- RisingPlatform만 상승하지 않고 초기 위치에 남아 있음

원인

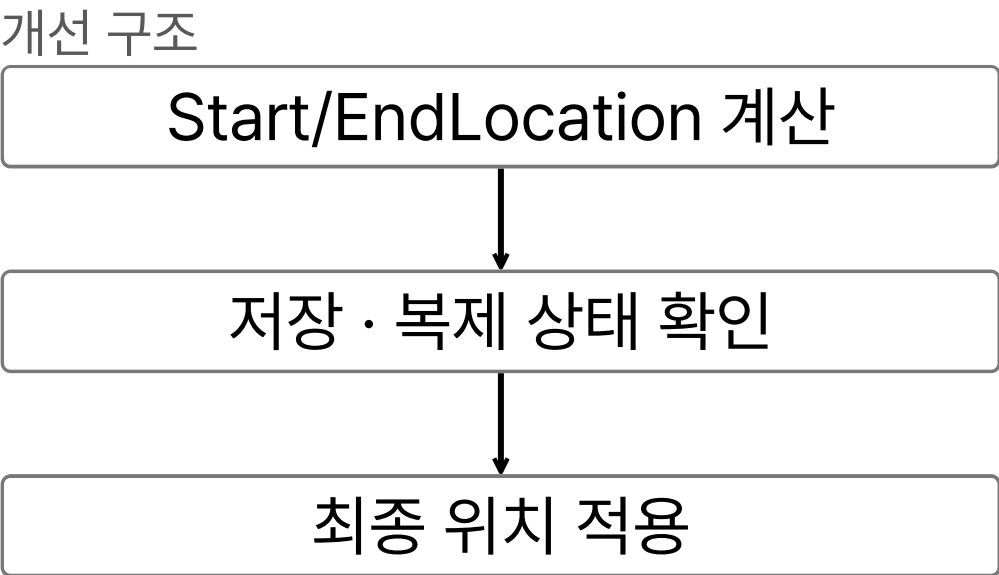
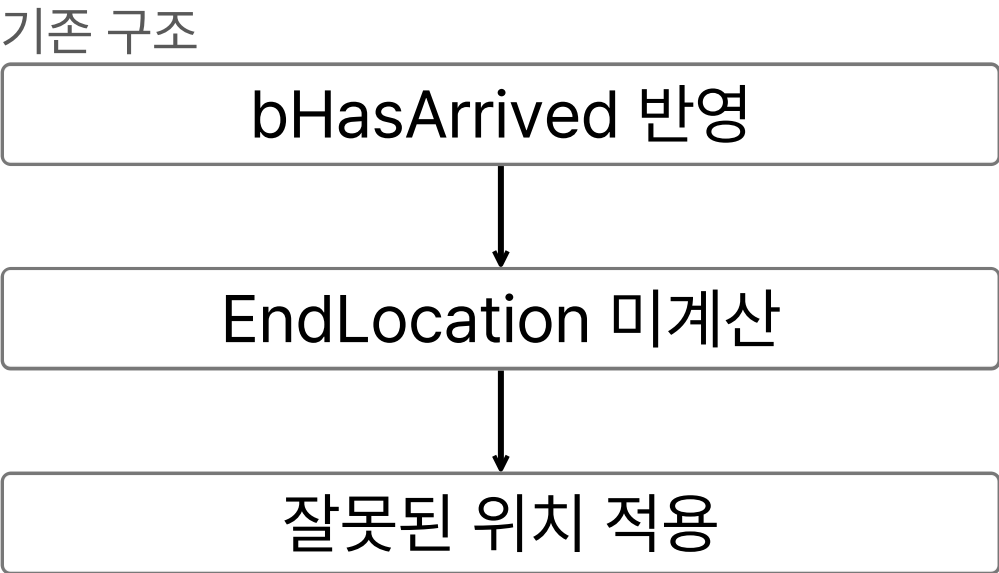
- **EndLocation이 초기화되지 않은 상태**에서 현재 위치를 기준으로 재계산되어 잘못된 위치가 최종 상태로 유지

해결

- BeginPlay()에서 StartLocation, EndLocation을 우선 계산
- 위치 계산을 먼저 완료하여 **Replication 적용 전**에 필요한 기준값을 보장
- 처리 순서를 **위치 계산 → 상태 확인 → 최종 위치 적용**으로 재구성
- bHasArrived == true이면 즉시 EndLocation으로 이동

결과

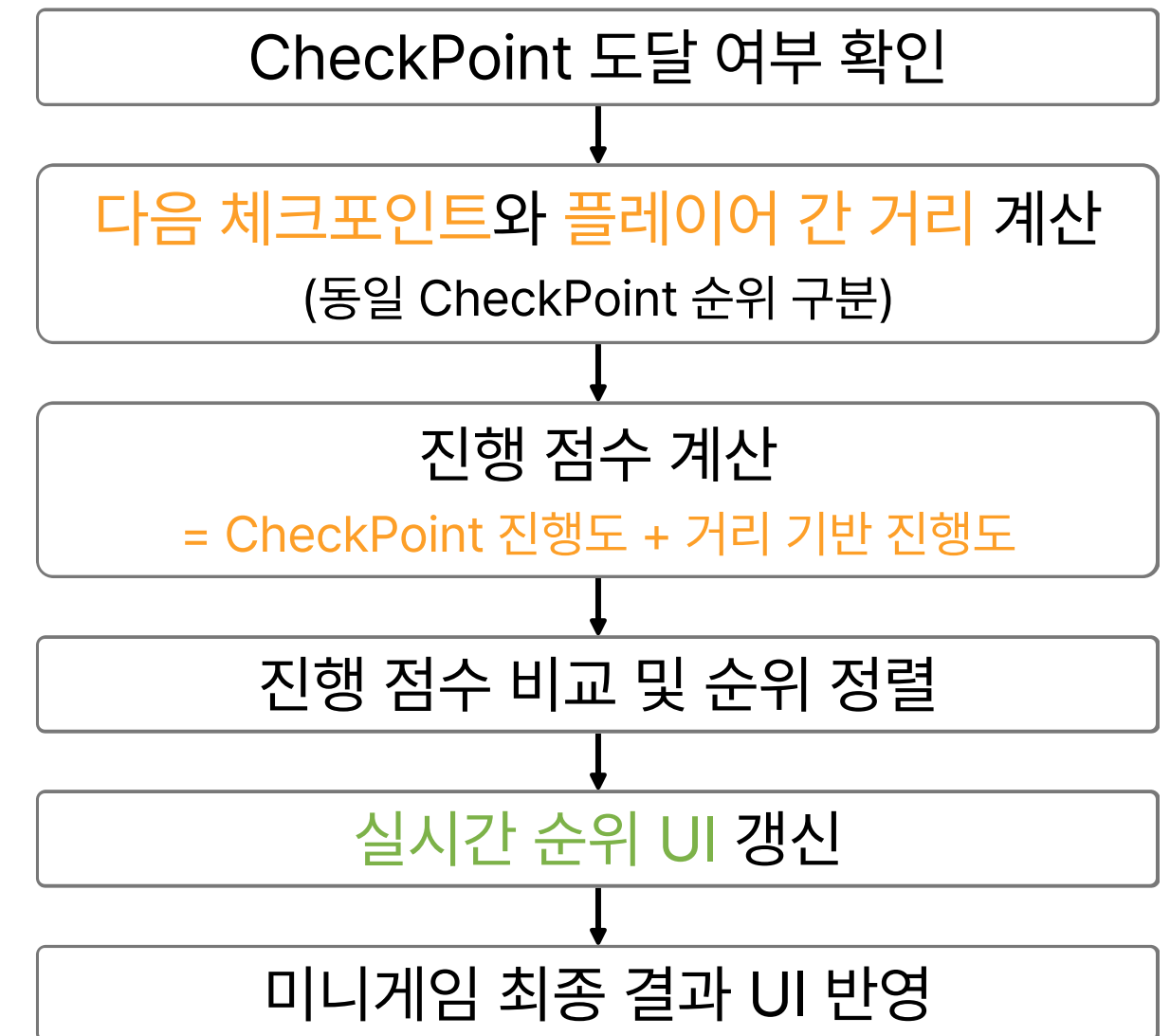
- **복제 적용 순서와 관계없이** 동일한 최종 위치 보장
- 모든 클라이언트에서 RisingPlatform 상태 동일 복원



Stage1 미니게임 플레이 화면



최종 결과 UI



- CheckPoint 진행도와 다음 CheckPoint까지의 거리로 진행 점수 계산
- 같은 CheckPoint 구간에서도 거리 기반 진행도로 순위를 구분
- 완주자는 도착 순위를 유지하고, 미완주자는 최종 진행 점수로 결과 순위 결정
- 순위 계산은 서버에서 수행하고, 체크포인트/순위 정보는 PlayerState로 동기화하며, 최종 결과 UI만 클라이언트에 전달

GameState 기반 Phase 상태 머신

Waiting	Surviving	FloorSinking	Finished
- 게임 시작 시 대기 5초 카운트다운 	- 플레이어 간 생존전 진행 - 전장 축소까지 남은 시간 표시 전장 축소 전 	- ArenaFloor 하강 시작 - 전장 축소 진행 전장 축소 중 	- 최후 생존자 결정 - 결과 UI 출력 

- Waiting → Surviving → FloorSinking → Finished의 4단계 Phase로 구성
- Surviving과 FloorSinking을 반복하며 최대 2회 전장을 축소
- 플레이어 탈락 시점에 생존 순위를 기록
- Phase 상태는 GameState에 복제하고, UI 반응은 PlayerController RPC로 분리
- 상태 관리와 UI 처리를 분리해 미니게임 흐름을 독립적으로 유지하도록 설계

```
enum class EArenaPattern : uint8
{
    Waiting,
    Surviving,
    FloorSinking,
    Finished
};
```

보스전 - 부활카운트

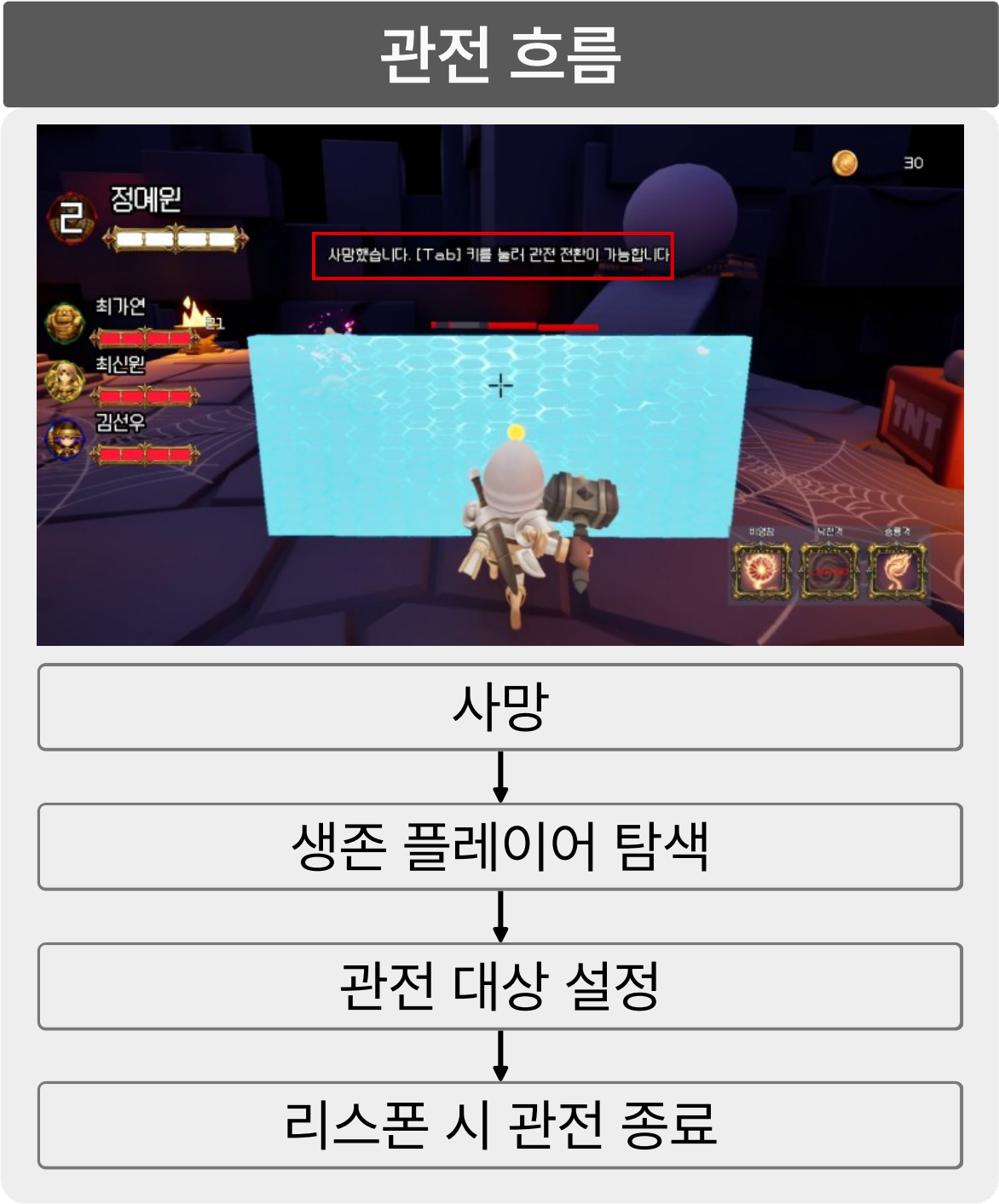


스테이지 - 플레이어 사망 화면



구분	일반 스테이지	미니게임	보스전
리스폰 시간	사망 횟수에 따라 증가, 최대값 제한	고정	고정
리스폰 기준	개인 사망 횟수	개인 진행도 및 사망 상태	팀 부활 횟수
상태 관리	PlayerState(개인)	PlayerState(개인)	GameState(공용)
체크포인트	공용	개인	없음
부활 조건	대기시간 종료	대기시간 종료	팀 부활 횟수 잔여

- 게임 모드마다 진행 규칙이 달라 GameMode에서 정책을 분리 구현
- 개인 상태는 PlayerState, 팀 공유 상태는 GameState로 분리 관리
- 일반 스테이지는 DeathCount 기반 지연 증가, 미니게임은 개인 체크포인트 기준, 보스전은 팀 기준으로 처리



- GameMode: 리스폰 규칙 | GameState: 공용 상태 | PlayerState: 개인 상태 | PlayerController: 관전

감사합니다

Thank You
